Task 1:

```
INCLUDE Irvine32.inc
.386
.model flat, stdcall
.stack 4096
ExitProcess proto, dwExitCode:dword

.data
	strArr   BYTE 9 DUP(?)
	digitArr   DD 8 DUP(?)
	msg     BYTE "Enter an 8 digit integer: ", 0
	msgSum        BYTE "Sum of first two digits: ", 0
	msgSub        BYTE "Subtraction of 3rd and 4th digits: ", 0
	msgMul        BYTE "Multiplication of 5th and 6th digits: ", 0
	msgDiv        BYTE "Division of 7th by 8th digit: ", 0
	sum     DD ?
	subtraction DD ?
	multiple   DD ?
	divide  DD ?

.code
main PROC

	mov edx, OFFSET msg
	call WriteString

	mov edx, OFFSET strArr
	mov ecx, 9
	call ReadString

	mov edx, OFFSET strArr
	call WriteString

	mov ecx, 8
	mov esi, OFFSET strArr
	mov edi, OFFSET digitArr
convertLoop:
	movzx eax, byte ptr [esi]
	sub eax, '0'
	mov [edi], eax
	inc esi
	add edi, 4
	loop convertLoop
```

```
mov esi, OFFSET digitArr

mov eax, [esi]
add eax, [esi+4]
mov sum, eax

mov eax, [esi+8]
sub eax, [esi+12]
mov subtraction, eax

mov eax, [esi+16]
mov ebx, [esi+20]
mul ebx
mov multiple, eax

mov eax, [esi+24]
mov ebx, [esi+28]
mov edx, 0
div ebx
mov divide, eax

mov edx, OFFSET msgSum
call WriteString
mov eax, sum
call WriteInt
call Crlf

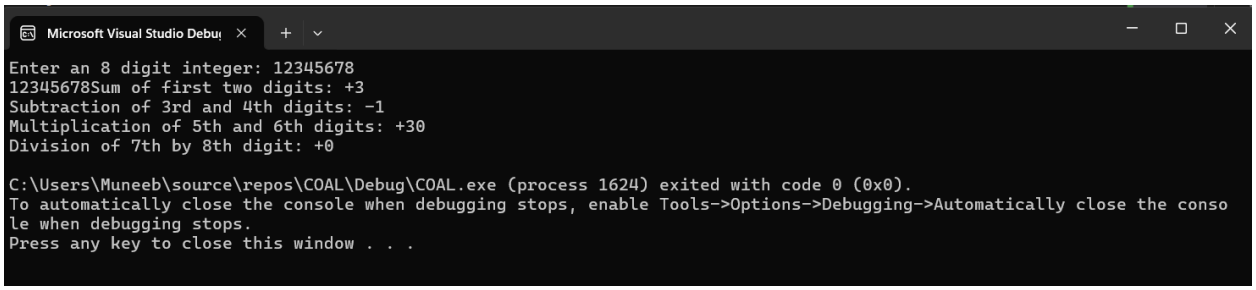
mov edx, OFFSET msgSub
call WriteString
mov eax, subtraction
call WriteInt
call Crlf

mov edx, OFFSET msgMul
call WriteString
mov eax, multiple
call WriteInt
call Crlf

mov edx, OFFSET msgDiv
call WriteString
mov eax, divide
call WriteInt
call Crlf
```

```
        invoke ExitProcess, 0
main ENDP

END main
```



Task 2:

```
INCLUDE Irvine32.inc
.386
.model flat, stdcall
.stack 4096
ExitProcess proto, dwExitCode:dword

.data
        asterisk byte "*",0
        msg byte "Enter number of rows: ",0
        space byte " ",0
        numRows DD ?

.code
main PROC

        mov edx, offset msg
        call WriteString
        call ReadInt
        mov numRows, eax


        push numRows
        call InvertedHalfStarPyramid
        call Crlf


        push numRows
        call StarPyramid
        call Crlf
```

```
        push numRows
        call DiamondPattern

        ; Exit program
        invoke ExitProcess, 0
main ENDP

;Inverted Half-Star Pyramid
InvertedHalfStarPyramid PROC
        push ebp
        mov ebp, esp
        mov ecx, [ebp+8]  ; Number of rows

outer:
        push ecx
        mov edx, offset asterisk

inner:
        call WriteString
        loop inner

        call Crlf
        pop ecx
        loop outer

        pop ebp
        ret 4
InvertedHalfStarPyramid ENDP

; Star Pyramid
StarPyramid PROC
        push ebp
        mov ebp, esp
        mov ecx, [ebp+8]  ; Number of rows
        mov esi, 1        ; Row index

printRow:
        push ecx

        ; Print leading spaces
        mov eax, [ebp+8]
        sub eax, esi
printSpace:
        cmp eax, 0
```

```
        jle endSpace
        mov edx, offset space
        call WriteString
        dec eax
        jmp printSpace
endSpace:

        ; Print stars (2*esi - 1)
        mov eax, esi
        shl eax, 1
        sub eax, 1
printStars:
        cmp eax, 0
        jle endStars
        mov edx, offset asterisk
        call WriteString
        dec eax
        jmp printStars
endStars:

        call Crlf
        inc esi
        pop ecx
        loop printRow

        pop ebp
        ret 4
StarPyramid ENDP

; Diamond Pattern
DiamondPattern PROC
        push ebp
        mov ebp, esp
        mov ecx, [ebp+8]  ; Number of rows (half of diamond height)
        mov esi, 1       ; Row index

        ; Upper Half
upperHalf:
        push ecx

        ; Print leading spaces
        mov eax, ecx
        dec eax
printUpperSpace:
```

```
        cmp eax, 0
        jle endUpperSpace
        mov edx, offset space
        call WriteString
        dec eax
        jmp printUpperSpace
endUpperSpace:

        ; Print stars (2*esi - 1)
        mov eax, esi
        shl eax, 1
        sub eax, 1
printUpperStars:
        cmp eax, 0
        jle endUpperStars
        mov edx, offset asterisk
        call WriteString
        dec eax
        jmp printUpperStars
endUpperStars:

        call Crlf
        inc esi
        pop ecx
        loop upperHalf

        ;Lower Half
        mov ecx, [ebp+8]
        dec ecx         ; Skip middle row
        mov esi, ecx

lowerHalf:
        push ecx

        ; Print leading spaces
        mov eax, [ebp+8]
        sub eax, esi
printLowerSpace:
        cmp eax, 0
        jle endLowerSpace
        mov edx, offset space
        call WriteString
        dec eax
        jmp printLowerSpace
```

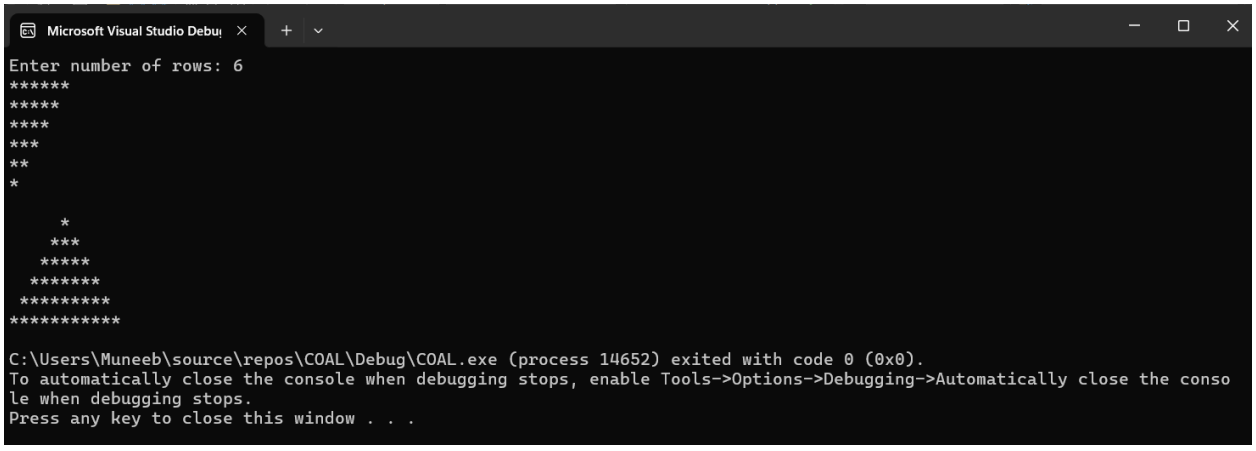
```
endLowerSpace:

        ; Print stars (2*esi - 1)
        mov eax, esi
        shl eax, 1
        sub eax, 1
printLowerStars:
        cmp eax, 0
        jle endLowerStars
        mov edx, offset asterisk
        call WriteString
        dec eax
        jmp printLowerStars
endLowerStars:

        call Crlf
        dec esi
        pop ecx
        loop lowerHalf

        pop ebp
        ret 4
DiamondPattern ENDP

END main
```



Task 3:

```
INCLUDE Irvine32.inc
.386
.model flat, stdcall
```

```
.stack 4096
ExitProcess proto, dwExitCode:dword

.data
	prompt DB "Enter 8 integers (one per line):", 0
	resultMsg DB "Most frequent element: ", 0
	numArr DD 8 DUP(?)
	countArr DD 8 DUP(0)
	maxCount DD 0
	maxElement DD 0

.code
main PROC

	mov edx, OFFSET prompt
	call WriteString
	call Crlf

	mov ecx, 8
	mov esi, OFFSET numArr

inputLoop:
	call ReadInt
	mov [esi], eax
	add esi, 4
	loop inputLoop

	; Step 2: Count occurrences of each element
	mov ecx, 8
	mov esi, OFFSET numArr

outerLoop:
	mov eax, [esi]
	mov edx, 0

	; Inner loop: count how many times EAX appears in numArr
	push ecx
	mov ecx, 8
	mov edi, OFFSET numArr

countLoop:
	cmp eax, [edi]
	jne skipCount
	inc edx
```

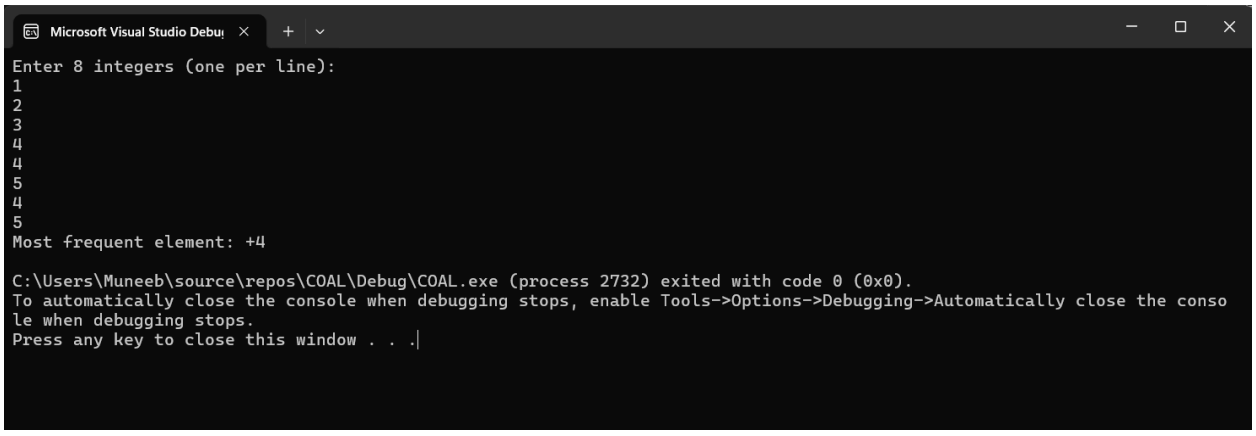
```
skipCount:
        add edi, 4
        loop countLoop


        cmp edx, [maxCount]
        jle notHigher
        mov [maxCount], edx
        mov [maxElement], eax
notHigher:
        pop ecx
        add esi, 4
        loop outerLoop

        ; Step 3: Print the most frequent element
        mov edx, OFFSET resultMsg
        call WriteString
        mov eax, [maxElement]
        call WriteInt
        call Crlf

        ; Exit program
        invoke ExitProcess, 0
main ENDP

END main
```



Task 4:

```
INCLUDE Irvine32.inc
.386
.model flat, stdcall
.stack 4096
ExitProcess proto, dwExitCode:dword
```

```
.data
        val1 DD 5
        val2 DD 10
        msg1 byte "Enter the first value: ",0
        msg2 byte "Enter the second value: ",0
.code
main PROC

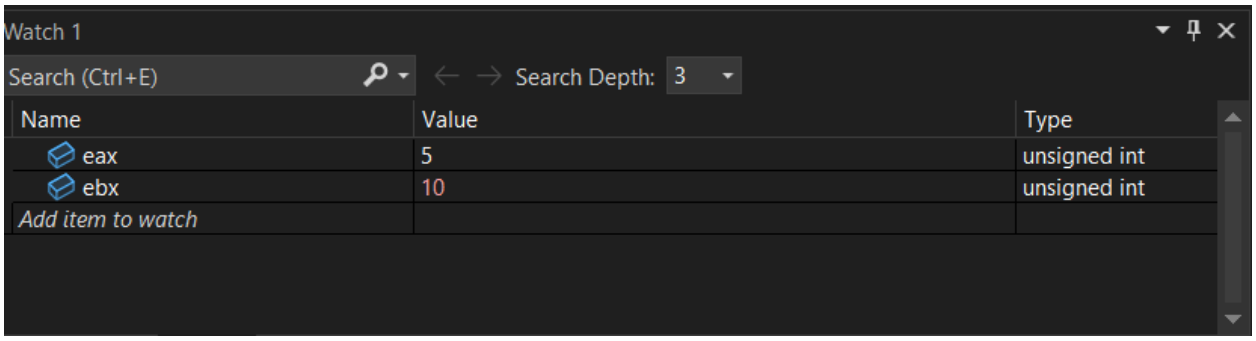
        mov edx, offset msg1
        call writestring
        call readint
        call writeint
        mov val1, eax

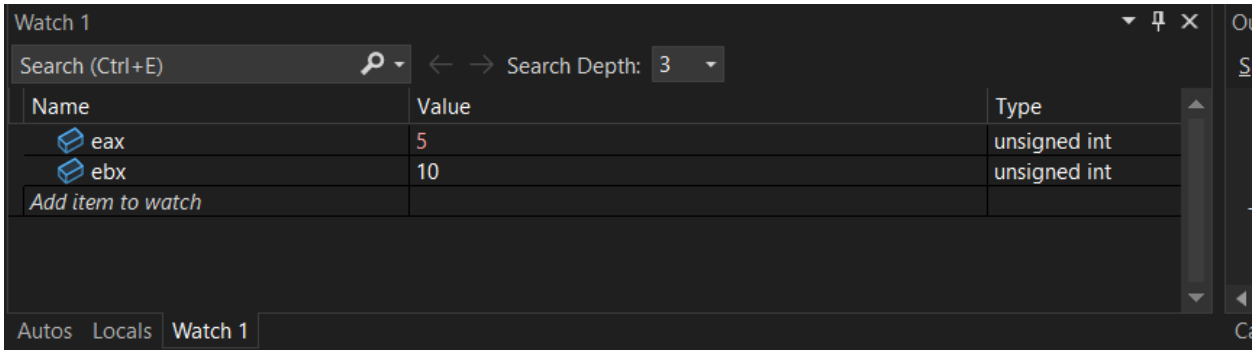
        mov edx, offset msg2
        call writestring
        call readint
        call writeint
        mov val2, eax

        mov eax, val1
        mov ebx, val2
        XOR eax, ebx
        XOR ebx,eax
        XOR ebx, eax
        xor eax, ebx

main ENDP

END main
```





Task 5:

```
INCLUDE Irvine32.inc
.386
.model flat, stdcall
.stack 4096
ExitProcess proto, dwExitCode:dword

.data
student1 DD 85, 90, 78, 92, 88
student2 DD 75, 80, 70, 60, 85
student3 DD 90, 95, 88, 91, 89
student4 DD 60, 70, 65, 55, 50
student5 DD 88, 82, 90, 85, 87

totalMarksPerSubject DD 5 DUP(100)

studentTotals   DD 5 DUP(?)
studentPercents  DD 5 DUP(?)
subjectHighs   DD 5 DUP(0)
subjectLows    DD 5 DUP(100)
subjectSums    DD 5 DUP(0)
subjectAverages  DD 5 DUP(?)

newline         BYTE 13,10,0
lblTotal        BYTE "Total Marks of Student ",0
lblPercent      BYTE "Percentage: ",0
lblHigh         BYTE "Highest Marks in Subject ",0
lblLow          BYTE "Lowest Marks in Subject ",0
lblAvg          BYTE "Average Marks in Subject ",0

.code
main PROC
        mov esi, offset student1
```

```
        mov eax, 0
        call ProcessStudent
        mov esi, offset student2
        mov eax, 1
        call ProcessStudent
        mov esi, offset student3
        mov eax, 2
        call ProcessStudent
        mov esi, offset student4
        mov eax, 3
        call ProcessStudent
        mov esi, offset student5
        mov eax, 4
        call ProcessStudent

        mov ecx, 5
        xor edi, edi
avg_loop:
        mov eax, subjectSums[edi*4]
        mov ebx, 5
        xor edx, edx
        div ebx
        mov subjectAverages[edi*4], eax
        inc edi
        loop avg_loop

        mov ecx, 5
        xor esi, esi
print_student:
        mov edx, offset lblTotal
        call WriteString
        mov eax, esi
        inc eax
        call WriteDec
        call Crlf

        mov eax, studentTotals[esi*4]
        call WriteInt
        call Crlf

        mov edx, offset lblPercent
        call WriteString
        mov eax, studentPercents[esi*4]
        call WriteInt
```

```
        call Crlf
        call Crlf

        inc esi
        loop print_student

        mov ecx, 5
        xor esi, esi
print_subjects:
        mov edx, offset lblHigh
        call WriteString
        mov eax, esi
        inc eax
        call WriteDec
        call Crlf

        mov eax, subjectHighs[esi*4]
        call WriteInt
        call Crlf

        mov edx, offset lblLow
        call WriteString
        mov eax, esi
        inc eax
        call WriteDec
        call Crlf

        mov eax, subjectLows[esi*4]
        call WriteInt
        call Crlf

        mov edx, offset lblAvg
        call WriteString
        mov eax, esi
        inc eax
        call WriteDec
        call Crlf

        mov eax, subjectAverages[esi*4]
        call WriteInt
        call Crlf
        call Crlf

        inc esi
```

```
        loop print_subjects

        invoke ExitProcess, 0
main ENDP

ProcessStudent PROC
        push esi
        push edi
        push ebx
        push ecx
        push edx

        xor ebx, ebx
        xor ecx, ecx

next_subj:
        mov edx, [esi + ebx*4]
        add ecx, edx
        add subjectSums[ebx*4], edx

        cmp edx, subjectHighs[ebx*4]
        jle skip_high
        mov subjectHighs[ebx*4], edx
skip_high:
        cmp edx, subjectLows[ebx*4]
        jge skip_low
        mov subjectLows[ebx*4], edx
skip_low:
        inc ebx
        cmp ebx, 5
        jl next_subj

        mov studentTotals[eax*4], ecx

        mov edi, eax
        mov eax, ecx
        mov edx, 0
        mov ebx, 100
        mul ebx
        mov ebx, 500
        div ebx
        mov studentPercents[edi*4], eax

        pop edx
```

```
        pop ecx
        pop ebx
        pop edi
        pop esi
        ret
ProcessStudent ENDP

END main
```

